

Análisis y Diseño de Algoritmos 2027-I

PCIC UNAM

Armando Castañeda Rojano

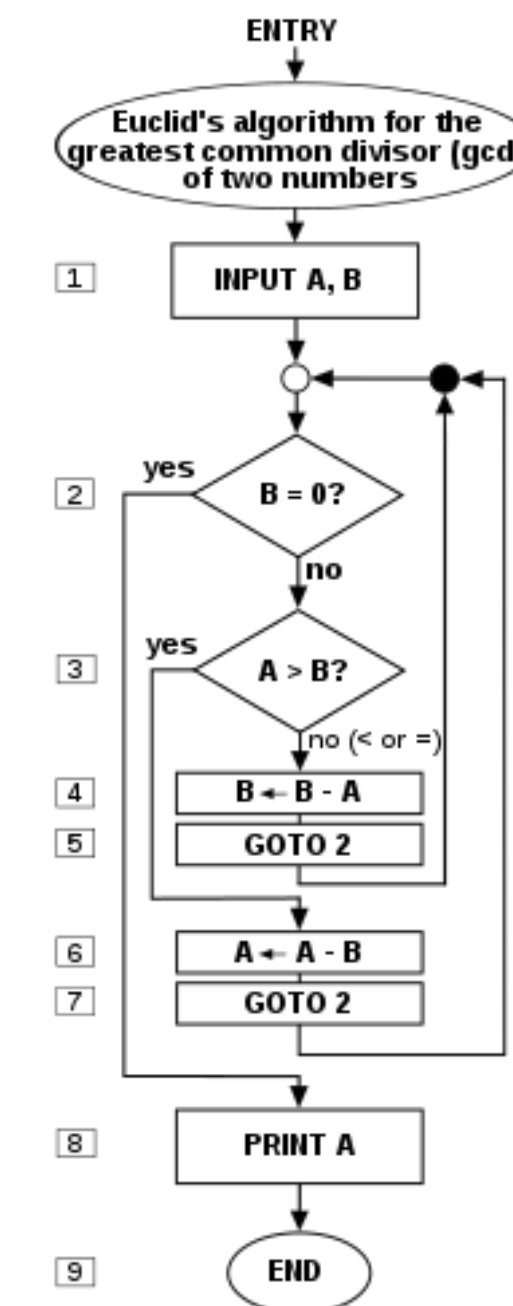
armando.castaneda@im.unam.mx

armando.castanedar@gmail.com

¿De qué va el curso?

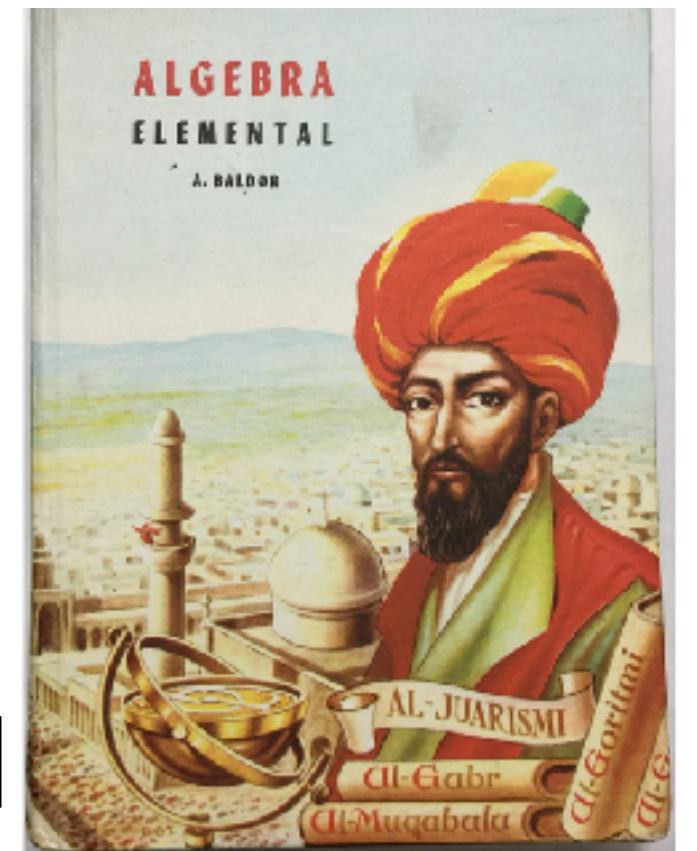
- Obviamente: **Algoritmos**.
- ¿Qué son? **Secuencia finita y bien definida de instrucciones para solucionar un problema.**
- Informalmente: Son como recetas de cocina (pero muy bien especificadas).

```
private static void SortEm(int[] ar) {  
    // We need a temporary storage area for when we want to  
    int temp;  
  
    // This is the code that does the bubble sort.  
    for (int i = ar.length - 1; i > 0; i--) {  
        for (int j = 0; j < i; j++) {  
            if (ar[j] > ar[j + 1]) {  
                temp = ar[j];  
                ar[j] = ar[j + 1];  
                ar[j + 1] = temp;  
            }  
        }  
    }  
}
```



¿Por qué estudiar algoritmos?

- Han existido desde hace mucho tiempo.
- Primero registros de algoritmos se encuentran en Babilonia (2500aC).
- **Algoritmo:** latinización de [Al-Khorezmi](#), del erudito persa Muhammad ibn Musa abu Djafar Al-Khorezmi (780 d.C.).
- Formalización: Máquinas de [Alan Turing](#) (1937).
- Estudio sistemático de los algoritmos es un área reciente.



¿Por qué estudiar algoritmos?

- Todo los días usamos los algoritmos: desde cocinar o armar una bicicleta hasta sumar, restar, o solucionar problemas computacionales sofisticados.
- La **algoritmia**, o ciencia de los algoritmos, es uno de los pilares de la computación.
- Con **relevancia a todas las áreas de conocimiento.**

¿Por qué estudiar algoritmos?

Contents [\[hide\]](#)

- 1 [Automated planning](#)
- 2 [Combinatorial algorithms](#)
 - 2.1 [General combinatorial algorithms](#)
 - 2.2 [Graph algorithms](#)
 - 2.2.1 [Graph drawing](#)
 - 2.2.2 [Network theory](#)
 - 2.2.3 [Routing for graphs](#)
 - 2.2.4 [Graph search](#)
 - 2.2.5 [Subgraphs](#)
 - 2.3 [Sequence algorithms](#)
 - 2.3.1 [Approximate sequence matching](#)
 - 2.3.2 [Selection algorithms](#)
 - 2.3.3 [Sequence search](#)
 - 2.3.4 [Sequence merging](#)
 - 2.3.5 [Sequence permutations](#)
 - 2.3.6 [Sequence alignment](#)
 - 2.3.7 [Sequence sorting](#)
 - 2.3.8 [Subsequences](#)
 - 2.3.9 [Substrings](#)
-
-

Algoritmos: desde cocinar o armar una bicicleta
hasta resolver problemas computacionales sofisticados.

Los algoritmos, es uno de los pilares de la

ciencia de la computación.

¿Por qué estudiar algoritmos?

Contents [hide]

- 1 Automated planning
- 2 Combinatorial algorithms
 - 2.1 General combinatorial algo
 - 2.2 Graph algorithms
 - 2.2.1 Graph drawing
 - 2.2.2 Network theory
 - 2.2.3 Routing for graphs
 - 2.2.4 Graph search
 - 2.2.5 Subgraphs
 - 2.3 Sequence algorithms
 - 2.3.1 Approximate sequen
 - 2.3.2 Selection algorithms
 - 2.3.3 Sequence search
 - 2.3.4 Sequence merging
 - 2.3.5 Sequence permutati
 - 2.3.6 Sequence alignment
 - 2.3.7 Sequence sorting
 - 2.3.8 Subsequences
 - 2.3.9 Substrings

3 Computational mathematics

- 3.1 Abstract algebra
- 3.2 Computer algebra
- 3.3 Geometry
- 3.4 Number theoretic algorithms
- 3.5 Numerical algorithms
 - 3.5.1 Differential equation solving
 - 3.5.2 Elementary and special functions
 - 3.5.3 Geometric
 - 3.5.4 Interpolation and extrapolation
 - 3.5.5 Linear algebra
 - 3.5.6 Monte Carlo
 - 3.5.7 Numerical integration
 - 3.5.8 Root finding
- 3.6 Optimization algorithms

4 Computational science

- 4.1 Astronomy
- 4.2 Bioinformatics
- 4.3 Geoscience
- 4.4 Linguistics
- 4.5 Medicine
- 4.6 Physics
- 4.7 Statistics

cinar o armar una bicicleta
nputacionales sofisticados.

no de los pilares de la

imiento.

¿Por qué estudiar algoritmos?

Contents [hide]

- 1 Automated planning
- 2 Combinatorial algorithms
 - 2.1 General combinatorial algo
 - 2.2 Graph algorithms
 - 2.2.1 Graph drawing
 - 2.2.2 Network theory
 - 2.2.3 Routing for graphs
 - 2.2.4 Graph search
 - 2.2.5 Subgraphs
 - 2.3 Sequence algorithms
 - 2.3.1 Approximate sequen
 - 2.3.2 Selection algorithms
 - 2.3.3 Sequence search
 - 2.3.4 Sequence merging
 - 2.3.5 Sequence permutati
 - 2.3.6 Sequence alignment
 - 2.3.7 Sequence sorting
 - 2.3.8 Subsequences
 - 2.3.9 Substrings

3 Computational mathematics

- 3.1 Abstract algebra
- 3.2 Computer algebra
- 3.3 Geometry
- 3.4 Number theoretic algorithms
- 3.5 Numerical algorithms
 - 3.5.1 Differential equation solving
 - 3.5.2 Elementary and special functions
 - 3.5.3 Geometric
 - 3.5.4 Interpolation and extrapolation
 - 3.5.5 Linear algebra
 - 3.5.6 Monte Carlo
 - 3.5.7 Numerical integration
 - 3.5.8 Root finding
- 3.6 Optimization algorithms

4 Computational science

- 4.1 Astronomy
- 4.2 Bioinformatics
- 4.3 Geoscience
- 4.4 Linguistics
- 4.5 Medicine
- 4.6 Physics
- 4.7 Statistics

5 Computer science

- 5.1 Computer architecture
 - 5.2 Computer graphics
 - 5.3 Cryptography
 - 5.4 Digital logic
 - 5.5 Machine learning and statistical classification
 - 5.6 Programming language theory
 - 5.6.1 Parsing
 - 5.7 Quantum algorithms
 - 5.8 Theory of computation and automata
- ## 6 Information theory and signal processing
- 6.1 Coding theory
 - 6.1.1 Error detection and correction
 - 6.1.2 Lossless compression algorithms
 - 6.1.3 Lossy compression algorithms
 - 6.2 Digital signal processing
 - 6.2.1 Image processing

7 Software engineering

8 Database algorithms

9 Distributed systems algorithms

- 9.1 Memory allocation and deallocation algorithms

10 Networking

11 Operating systems algorithms

¿Qué vamos a hacer en el curso?

- ¿Memorizar muchos algoritmos?
- ¿Aprendernos TODOS los algoritmos?

¿Qué vamos a hacer en el curso?

- ¿Memorizar muchos algoritmos?
- ¿Aprendernos TODOS los algoritmos?

NO!!



¿Qué vamos a hacer en el curso?

- ¿Memorizar muchos algoritmos?
- ¿Aprendernos TODOS los algoritmos?

NO!!

- Este curso es para **ENTENDER**.
- Técnicas para el análisis.
- Técnicas para el diseño.
- Herramientas muy valiosas en cualquier área de la computación.



Análisis de Algoritmos

- Buscamos entender por qué funcionan los algoritmos.
- Esencialmente buscamos argumentar dos propiedades:
 - Termina con cualquier entrada (*terminación*).
 - Calcula la salida correcta para toda entrada (*validez*).
- Otras propiedades:
 - Tiempo de ejecución (*complejidad de tiempo*).
 - Cantidad de memoria que usa (*complejidad de espacio*).
- Técnicas usadas para el análisis de **cualquier** tipo de algoritmo.



Ejemplo: Buscar en elemento en un arreglo

```
Alg. Busca(int[] A, int n, int x):  
  for int c = 1 up to n do  
    if A[c] == x then  
      return c  
    end for  
  return NULL  
end Busca
```

Ejemplo: Buscar un elemento en un arreglo

```
Alg. Busca(int[] A, int n, int x):  
  for int c = 1 up to n do  
    if A[c] == x then  
      return c  
  end for  
  return NULL  
end Busca
```

- Termina porque el for ejecuta a lo más n iteraciones.
- Si x está en A , devuelve una posición en la que está, pues revisa cada entrada en el arreglo. De otra forma devuelve *NULL*.
- El *for* ejecuta a lo más n iteraciones, y entonces se ejecutan a lo más dn instrucciones, para algún entero d . Por lo tanto, el algoritmo corre en tiempo polinomial (de hecho, lineal).
- Usa una variable adicional a la entrada (entero c) durante toda la ejecución. Por lo tanto, el algoritmo usa espacio de trabajo constante.

Diseño de Algoritmos

- No podemos aprender todos los algoritmos ... pero sí podemos aprender **paradigmas/técnicas** de diseño:
 - Algoritmos voraces (greedy).
 - Divide y vencerás.
 - Programación dinámica.
 - Algoritmos aleatorios.
 - ...
- Son una **tecnología** para solucionar problemas en diversos ámbitos.



Buscamos eficiencia!

- Nos interesan algoritmos **eficientes**.
- **Eficiente = Tiempo polinomial** (a grandes rasgos).
- El algoritmo de búsqueda de un elemento es eficiente pues corre en tiempo lineal.
- Existen algoritmos ineficientes.
- Existen problemas para los que **no existen algoritmos eficientes**.
- Diferencia fundamental.

Ejemplo 1: Ruta más corta

```
Alg. Ruta(Mapa M, #ciudades n, ciudad s, ciudad t):  
  path = NULL  
  for int x = 2 to n do  
    for each path' = u1, u2, ... ux do  
      if path' es un s-t camino then  
        if length(path') < length(path) then  
          path = path'  
        end if  
      end if  
    end for  
  end for  
  return path  
end Ruta
```



Ejemplo 1: Ruta más corta

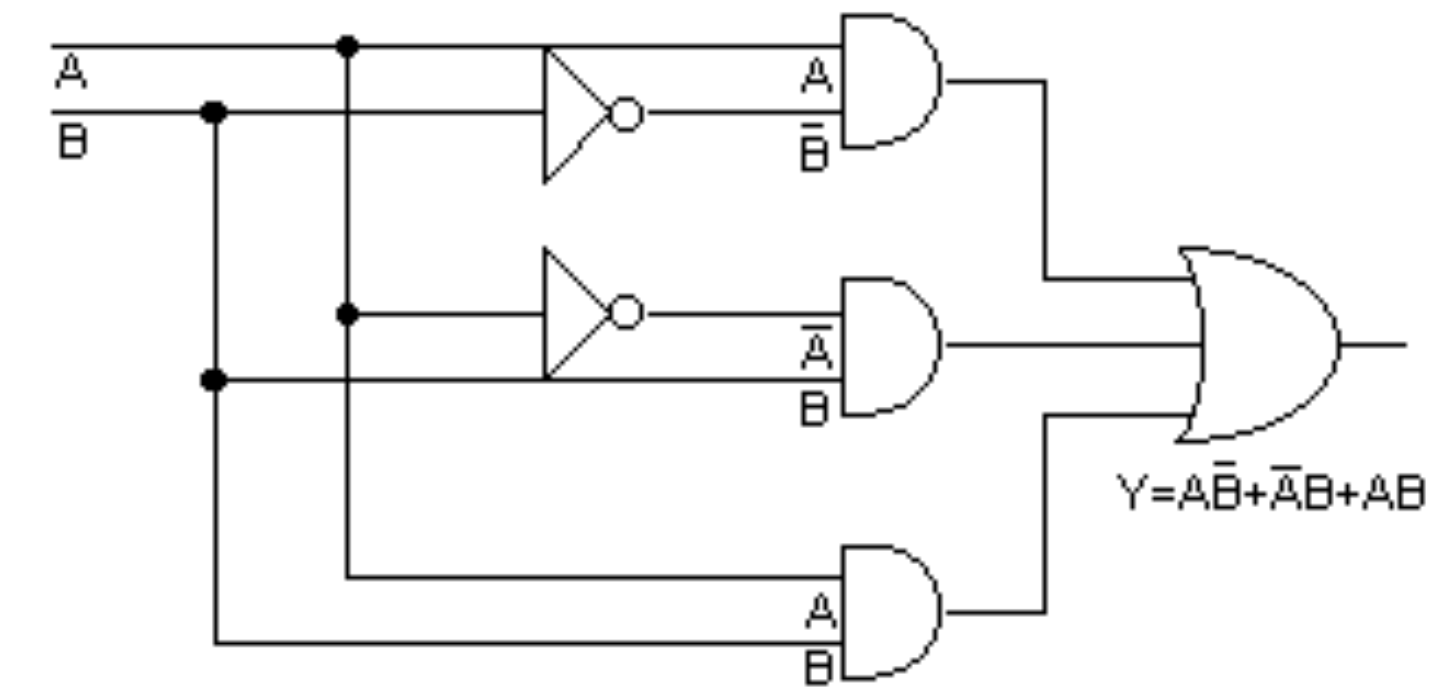
```
Alg. Ruta(Mapa M, #ciudades n, ciudad s, ciudad t):  
  path = NULL  
  for int x = 2 to n do  
    for each path' = u1, u2, ... ux do  
      if path' es un s-t camino then  
        if length(path') < length(path) then  
          path = path'  
    end for  
  return path  
end Ruta
```



- Se prueban más de $n!$ caminos.
- $30! = 2.652529e32$.
- El algoritmo es correcto?
- Sí ... pero no sirve de nada.
- Sin embargo, existen algoritmos eficientes para este problema.

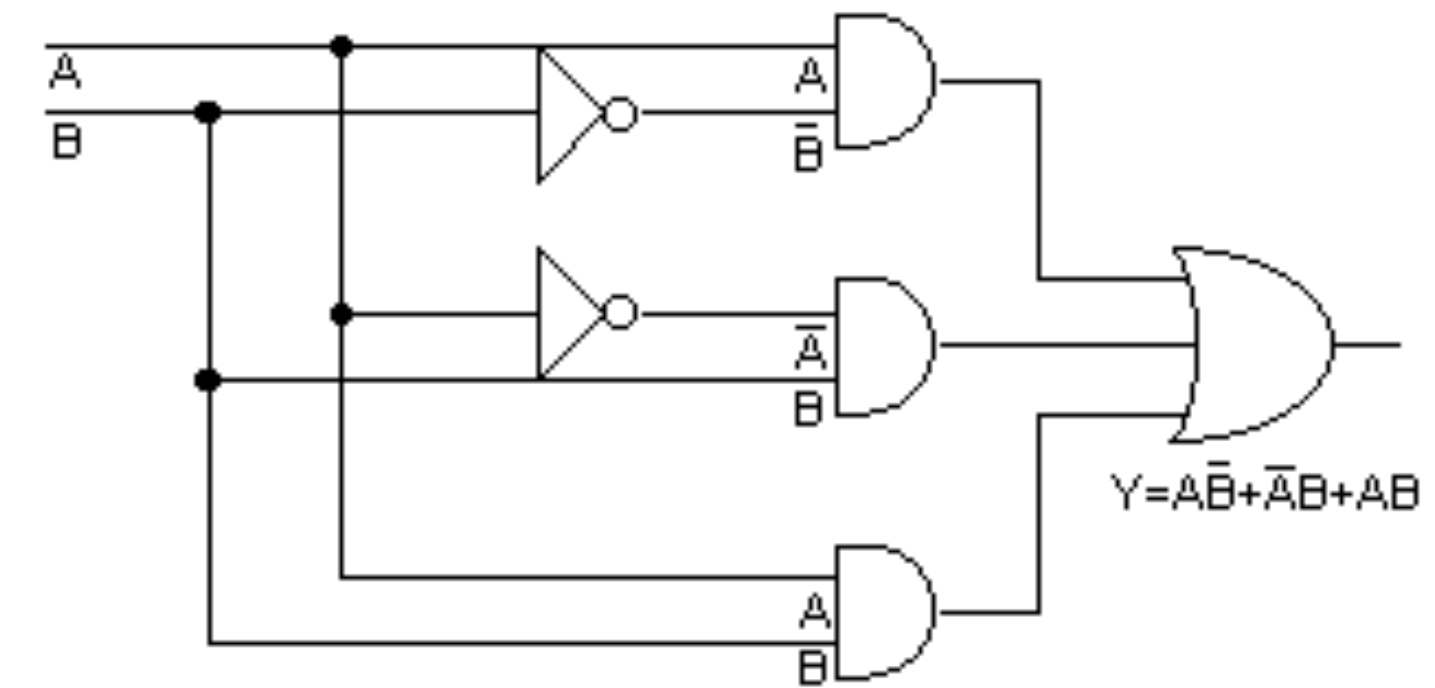
Ejemplo 2: Satisfacibilidad de circuitos

```
Alg. Satisfacibilidad(Circuito C, #entradas n):  
  for each entrada e1, e2, ... en do  
    if la salida de C es true then  
      return true  
    end for  
  return false  
end Satisfacibilidad
```



Ejemplo 2: Satisfacibilidad de circuitos

```
Alg. Satisfacibilidad(Circuito C, #entradas n):  
  for each entrada e1, e2, ... en do  
    if la salida de C es true then  
      return true  
    end for  
  return false  
end Satisfacibilidad
```



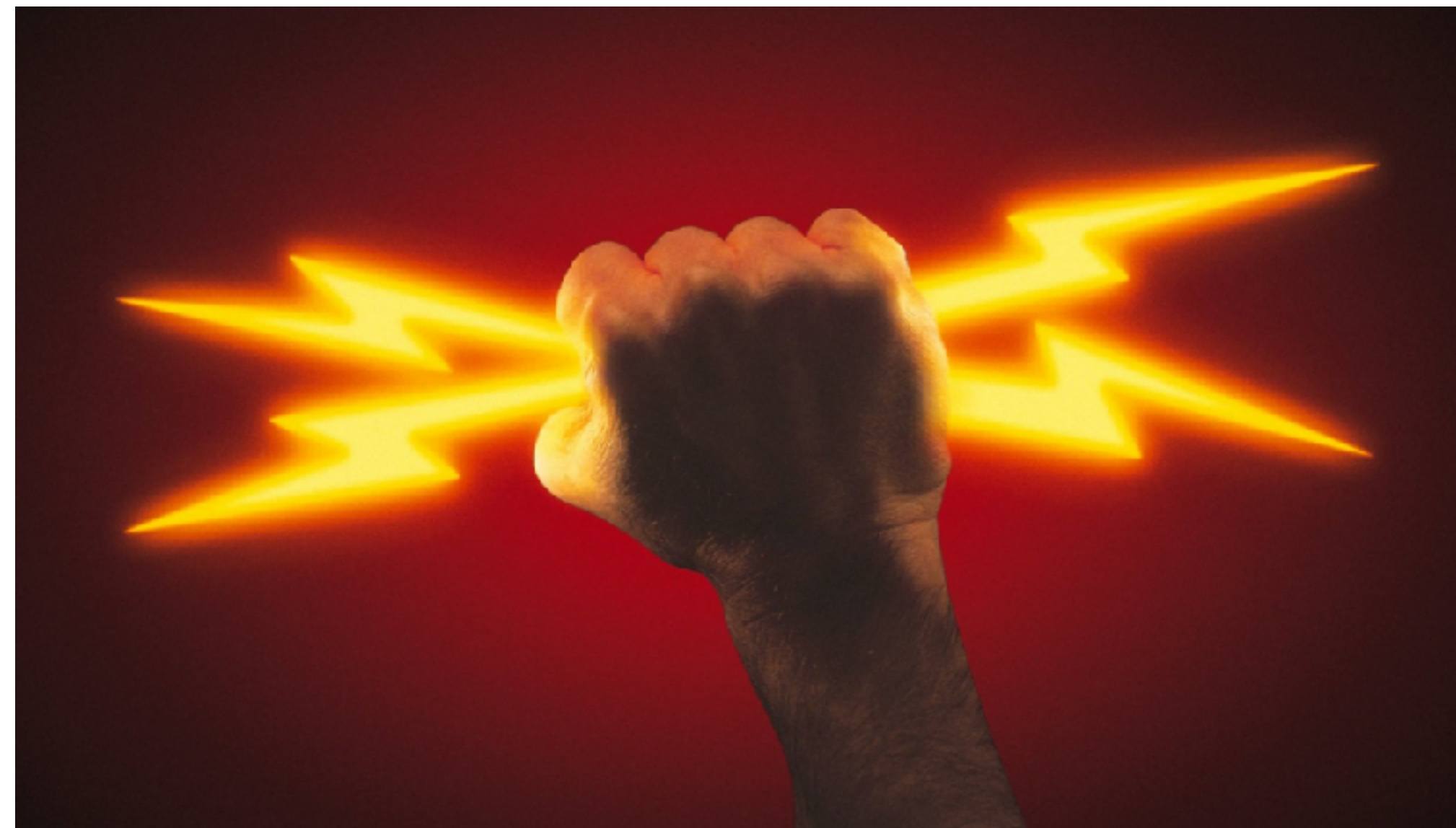
- Se pueden probar hasta 2^n entradas.
- $2^{256} = 1.157921e77$.
- Nadie sabe si existen algoritmos eficientes para este problema.
- Pregunta del millón: $P = NP$?

Se puede poner peor ...

- Existen problemas que **no tienen soluciones eficientes**.
 - Ejemplo: Ajedrez y Go.
- Existen problemas que **simplemente no se pueden solucionar**.
 - Ejemplos: verificación de programas (problema de la detención) y raíces enteras de polinomios.
- Importante entender esto: Algunas veces tenemos que vivir con soluciones aproximadas/parciales/probabilísticas.

El curso busca:

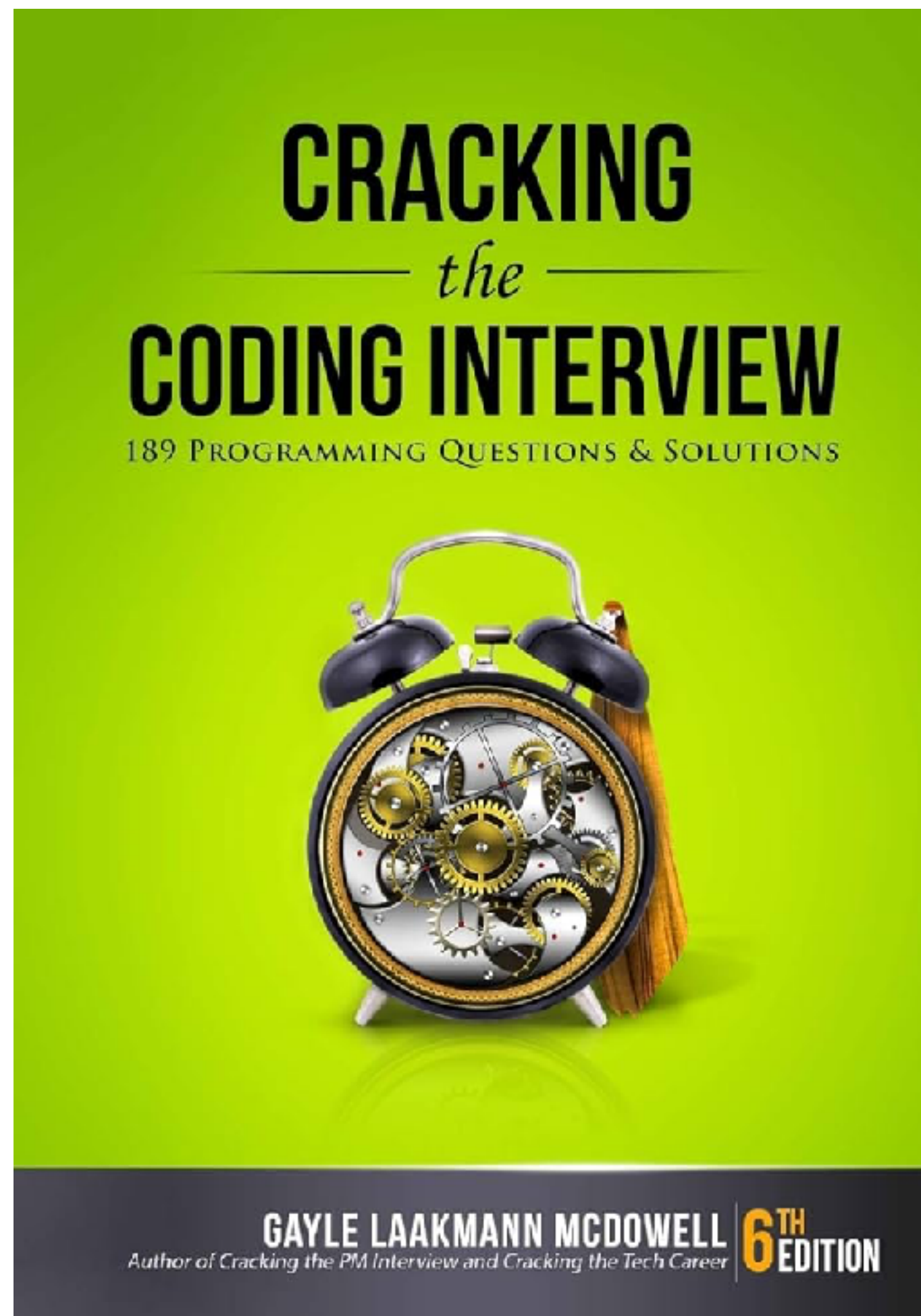
- Desarrollar habilidades para **entender** los algoritmos.
- Desarrollar habilidades para **crear** nuevos algoritmos **eficientes**.



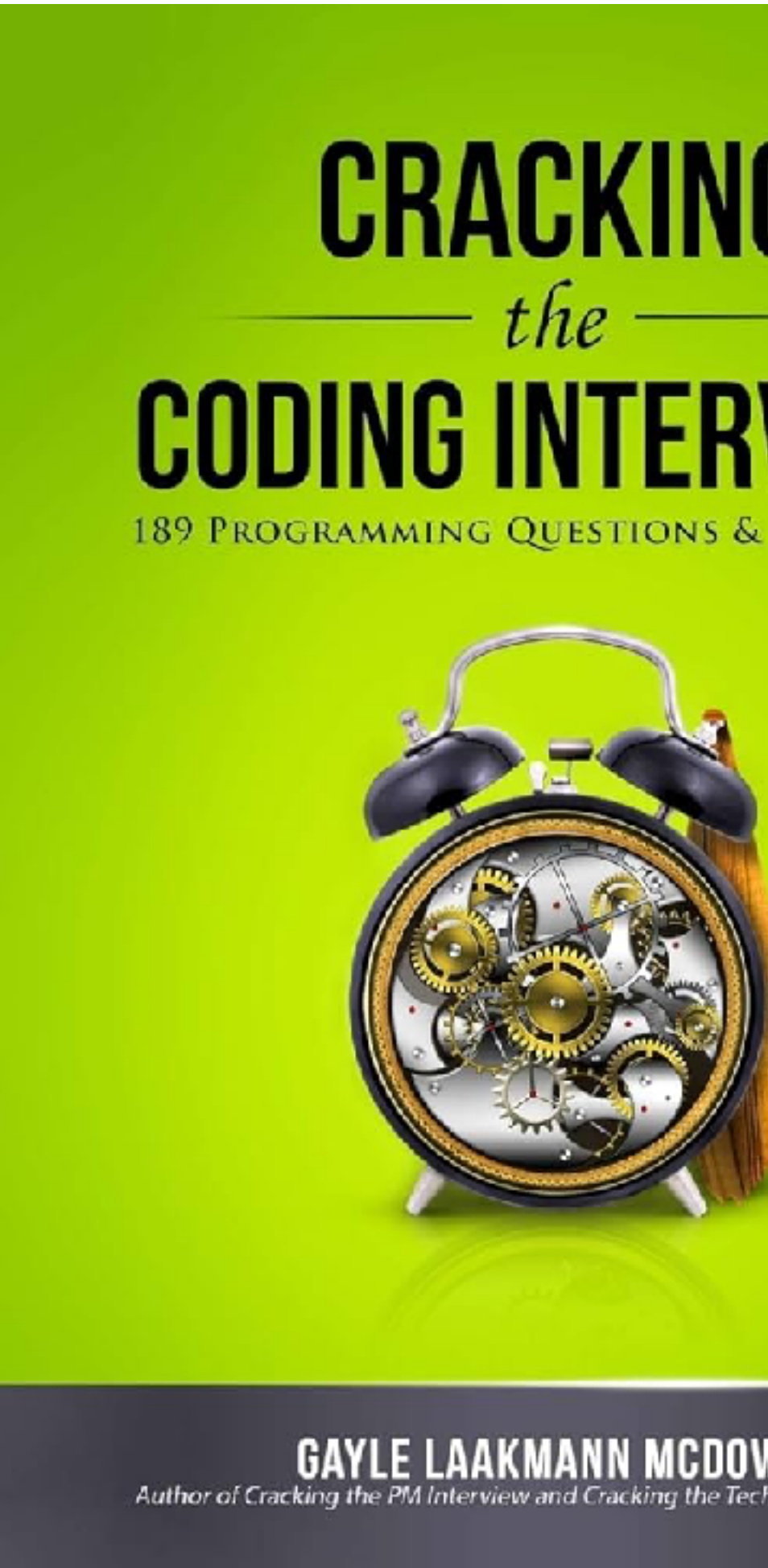
Solo a los de teoría les importa, no?



Una llave para entrar a compañías “*TOP*”



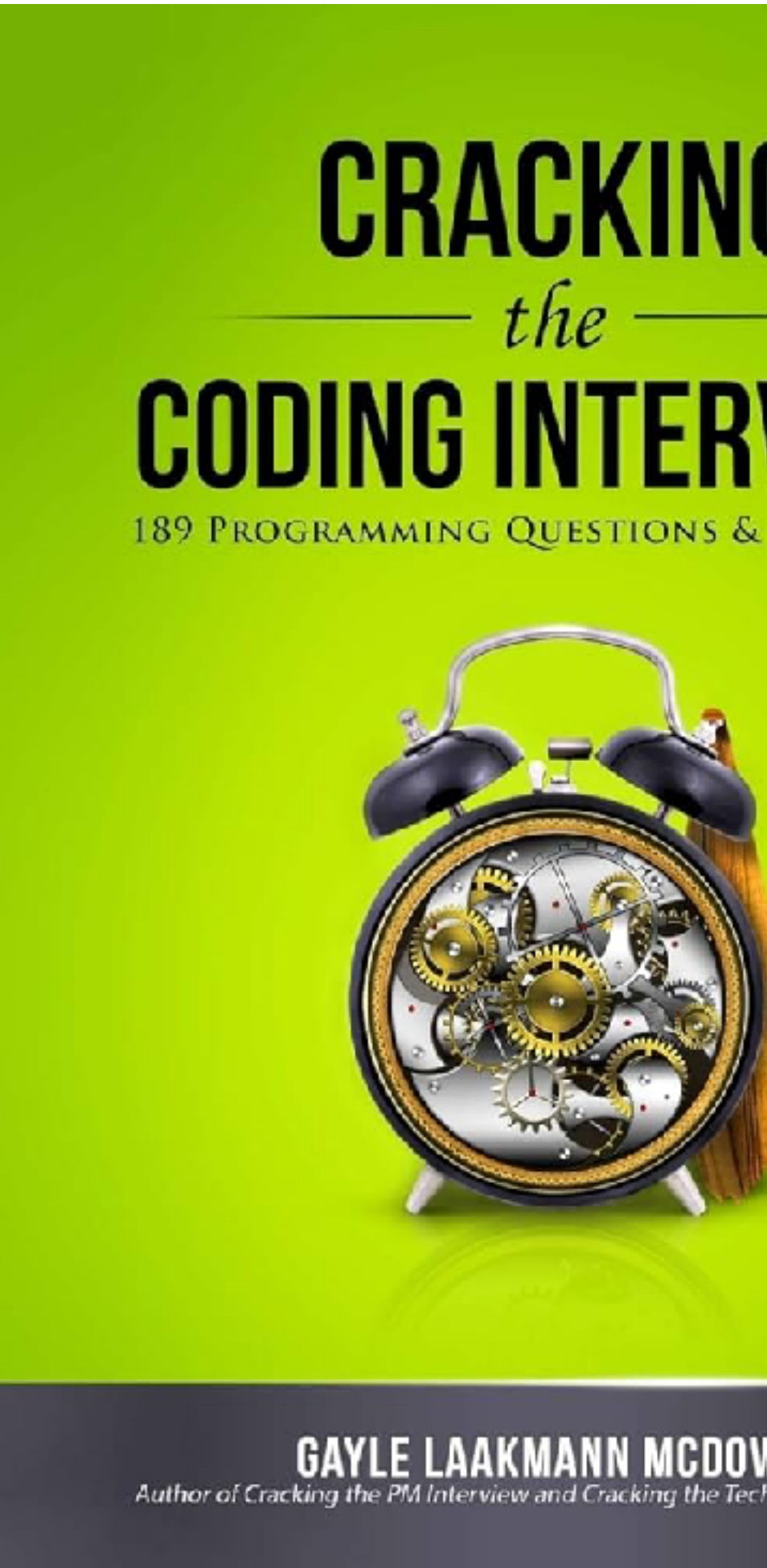
Una llave para entrar a compañías “TOP”



Introduction

Introduction	2
I. The Interview Process.	4
Why?_____	4
How Questions are Selected.	6
It's All Relative.	7
Frequently Asked Questions.	7
II. Behind the Scenes_____	8
The Microsoft Interview.	9
The Amazon Interview.	10
The Google Interview.	10
The Apple Interview.	11
The Facebook Interview.	12
The Palantir Interview.	13
III. Special Situations.	15
Experienced Candidates	15

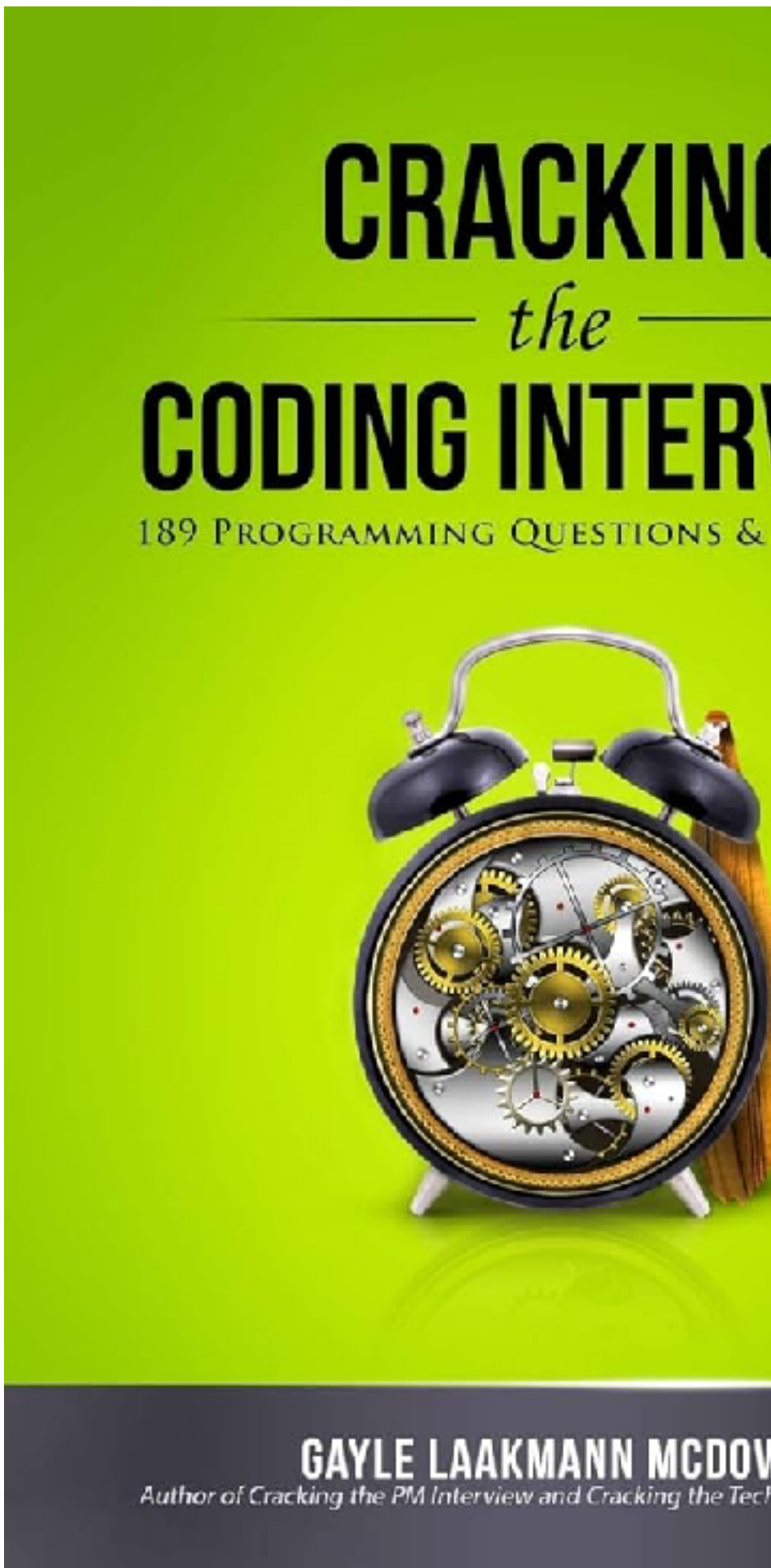
Una llave para entrar a compañías “TOP”



Introduction

Introduction	2
I. The Interview Process.	4
Why?	4
How Questions are Selected.	6
Interview Preparation Grid.	32
II. Behavioral Questions.	32
Know Your Technical Projects.	33
Responding to Behavioral Questions.	34
So, tell me about yourself.	38
III. BigO.	38
An Analogy.	38
Time Complexity.	38
Space Complexity.	40
Drop the Constants.	41
Drop the Non-Dominant Terms.	42

Una llave para entrar a compañías “TOP”



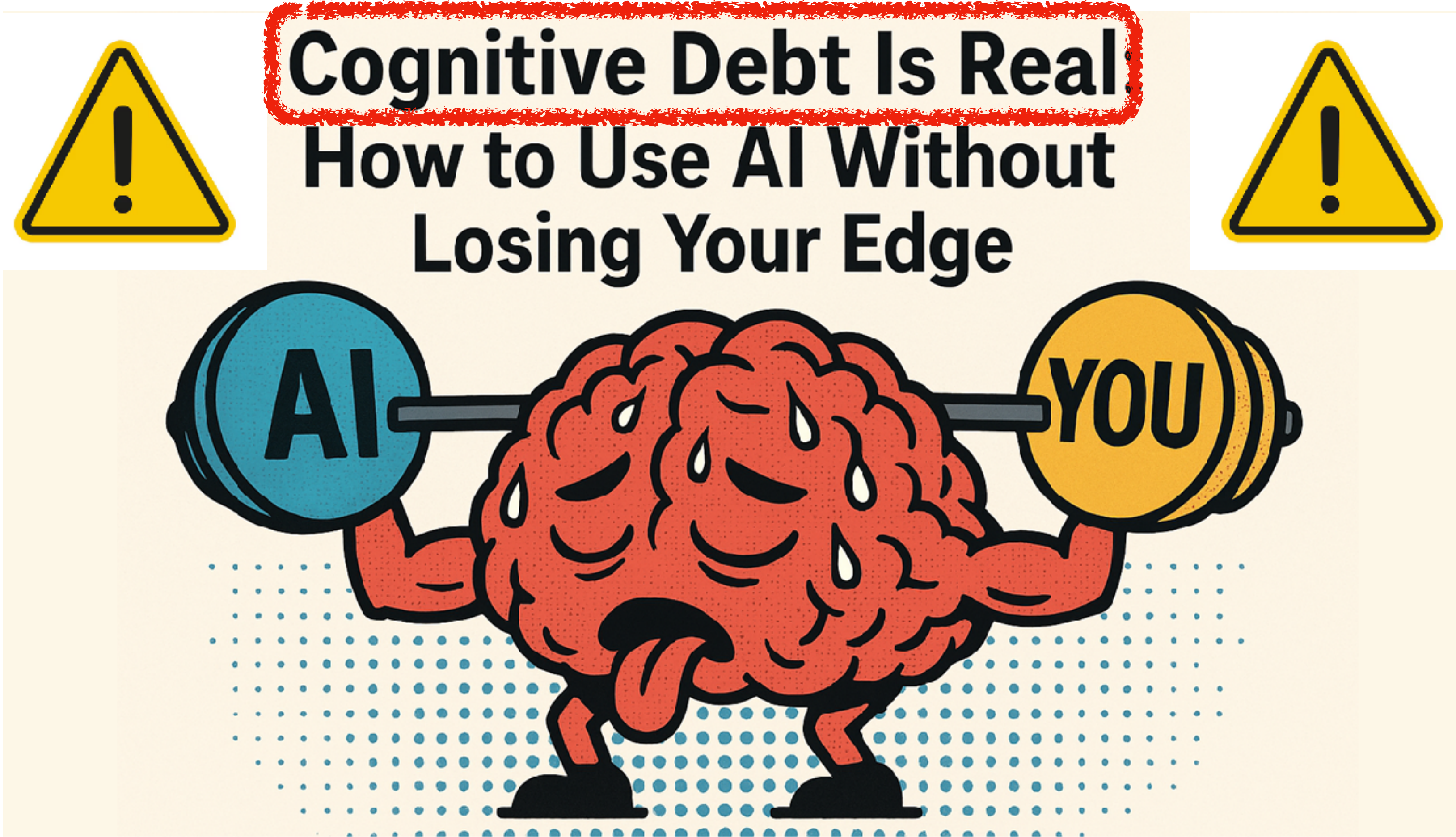
Introduction

Introduction	2
I. The Interview Process.	4
Why?	4
How Questions are Selected.	6
Interview Preparation Grid.	32
V. Behavioral Questions.	32
IX. Interview Questions.	87
Data Structures.	88
Chapter 1 Arrays and Strings.	88
Hash Tables.	88
ArrayList & Resizable Arrays.	89
StringBuilder.	89
Chapter 2 Linked Lists.	92
Creating a Linked List.	92
Deleting a Node from a Singly Linked List.	93
The "Runner" Technique.	93
Recursive Problems.	93

Pues usamos LLM's, no?



Pues usamos LLM's, no?



Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task[△]

Nataliya Kosmyna¹
MIT Media Lab
Cambridge, MA

Eugene Hauptmann
MIT
Cambridge, MA

Ye Tong Yuan
Wellesley College
Wellesley, MA

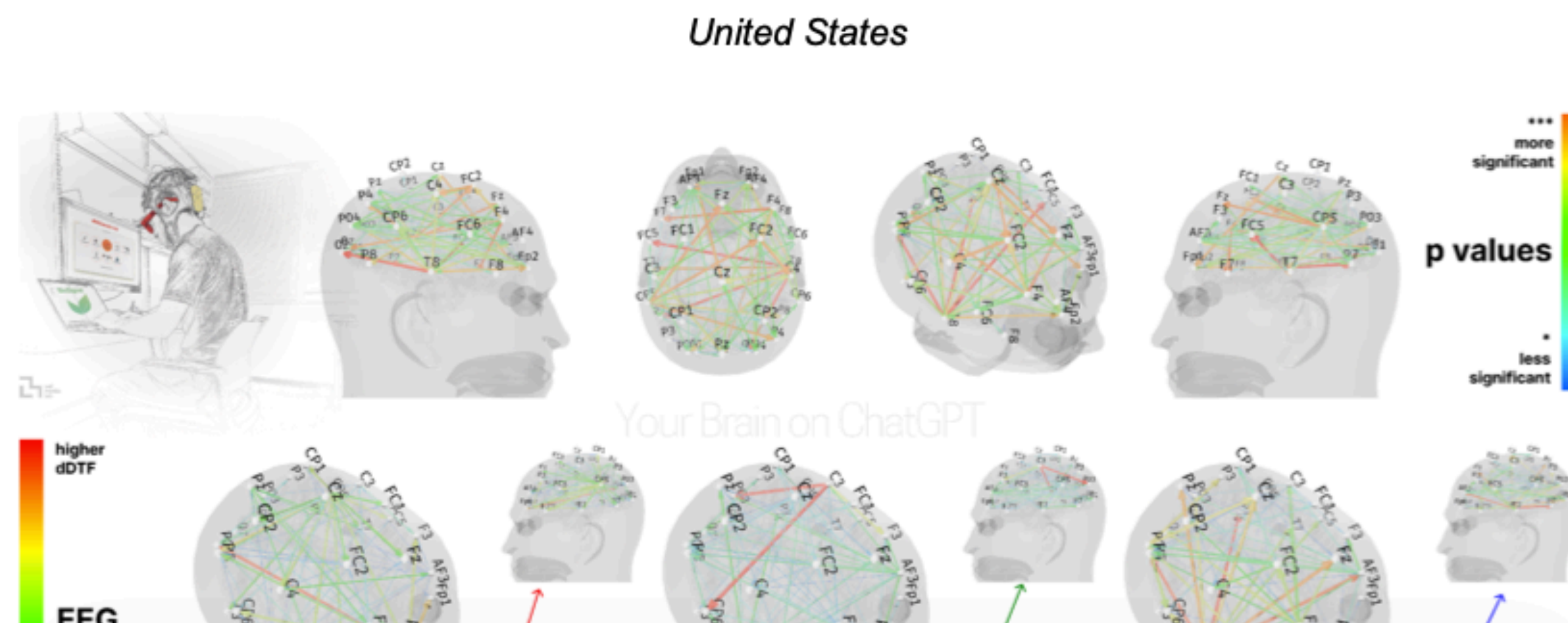
Jessica Situ
MIT
Cambridge, MA

Xian-Hao Liao
Mass. College of Art
and Design (MassArt)
Boston, MA

Ashly Vivian Beresnitzky
MIT
Cambridge, MA

Iris Braunstein
MIT
Cambridge, MA

Pattie Maes
MIT Media Lab
Cambridge, MA

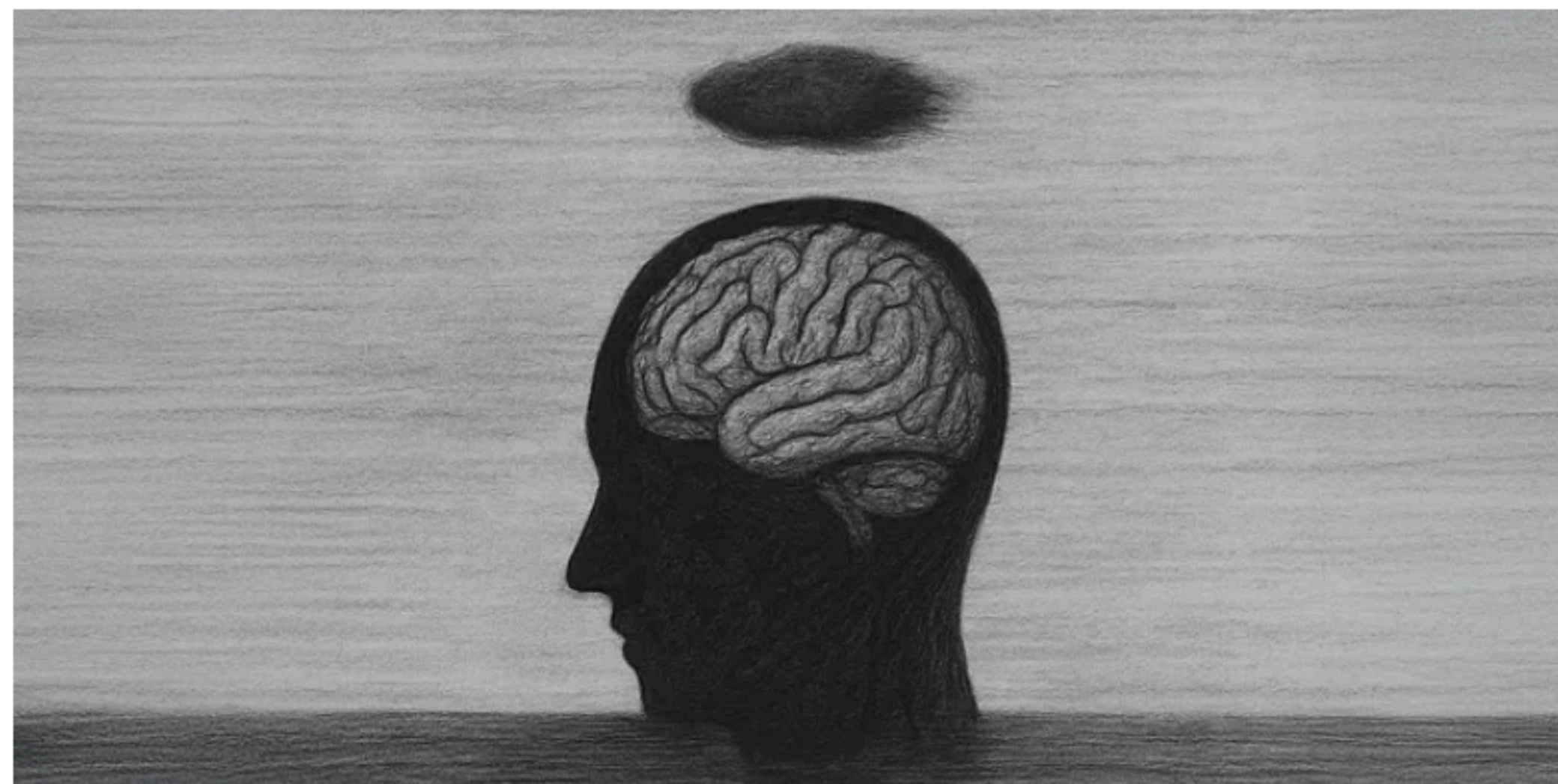


Bootcamp



From idea to product, one lesson at a time. To submit your story: <https://tinyurl.com/bootspub1>

[Follow publication](#)



Cognitive debt: What I learned from MIT's paper on AI and brain atrophy



Jake Miller

Follow

4 min read · Jun 23, 2025



Bootcamp



From idea
lesson at a
submit yo
<https://tinypub1>

Follow pub

Conclusion: Use AI But Don't Let It Use You

I'm not anti-AI. I use it all the time. But I use it *after* I've thought. After I've questioned. After I've worked the problem a little myself.

Because I still believe in the value of the work. I believe in the friction that learning creates. And I believe that anything worth knowing is worth struggling through.

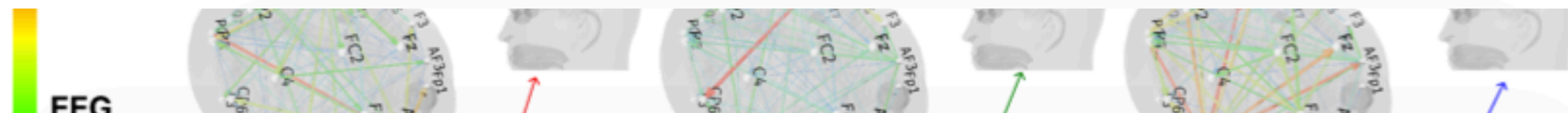
If AI makes me faster, great. But if it makes me dull, what's the point?



Jake Miller

Follow

4 min read · Jun 23, 2025



Bootcamp



From idea
lesson at a
submit yo
<https://tinypub1>

Follow pub

Conclusion: Use AI But Don't Let It Use You

I'm not anti-AI. I use it all the time. But I use it *after* I've thought. After I've questioned. After I've worked the problem a little myself.

Because I still believe in the value of the work. I believe in the friction that learning creates. And I believe that anything worth knowing is worth struggling through.

If AI makes me faster, great. But if it makes me dull, what's the point?



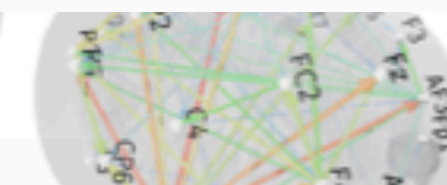
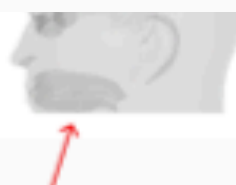
Jake Miller

Follow

4 min read · Jun 23, 2025



EEG



Bootcamp



From idea
lesson at a
submit yo
<https://tinypub1>

Follow pub

Conclusion: Use AI But Don't Let It Use You

I'm not anti-AI. I use it all the time. But I use it *after* I've thought. After I've questioned. After I've worked the problem a little myself.

Because I still believe in the value of the work. I believe in the friction that learning creates. And I believe that anything worth knowing is worth struggling through.

If AI makes me faster, great. But if it makes me dull, what's the point?



Jake Miller

Follow

4 min read · Jun 23, 2025



Bootcamp



From idea
lesson at a
submit yo
<https://tinypub1>

Follow pub

Conclusion: Use AI But Don't Let It Use You

I'm not anti-AI. I use it all the time. But I use it *after* I've thought. After I've questioned. After I've worked the problem a little myself.

Because I still believe in the value of the work. I believe in the friction that learning creates. And I believe that anything worth knowing is worth struggling through.

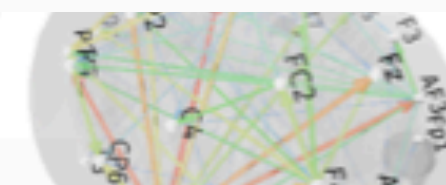
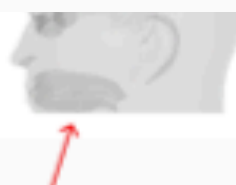
If AI makes me faster, great. But if it makes me dull, what's the point?



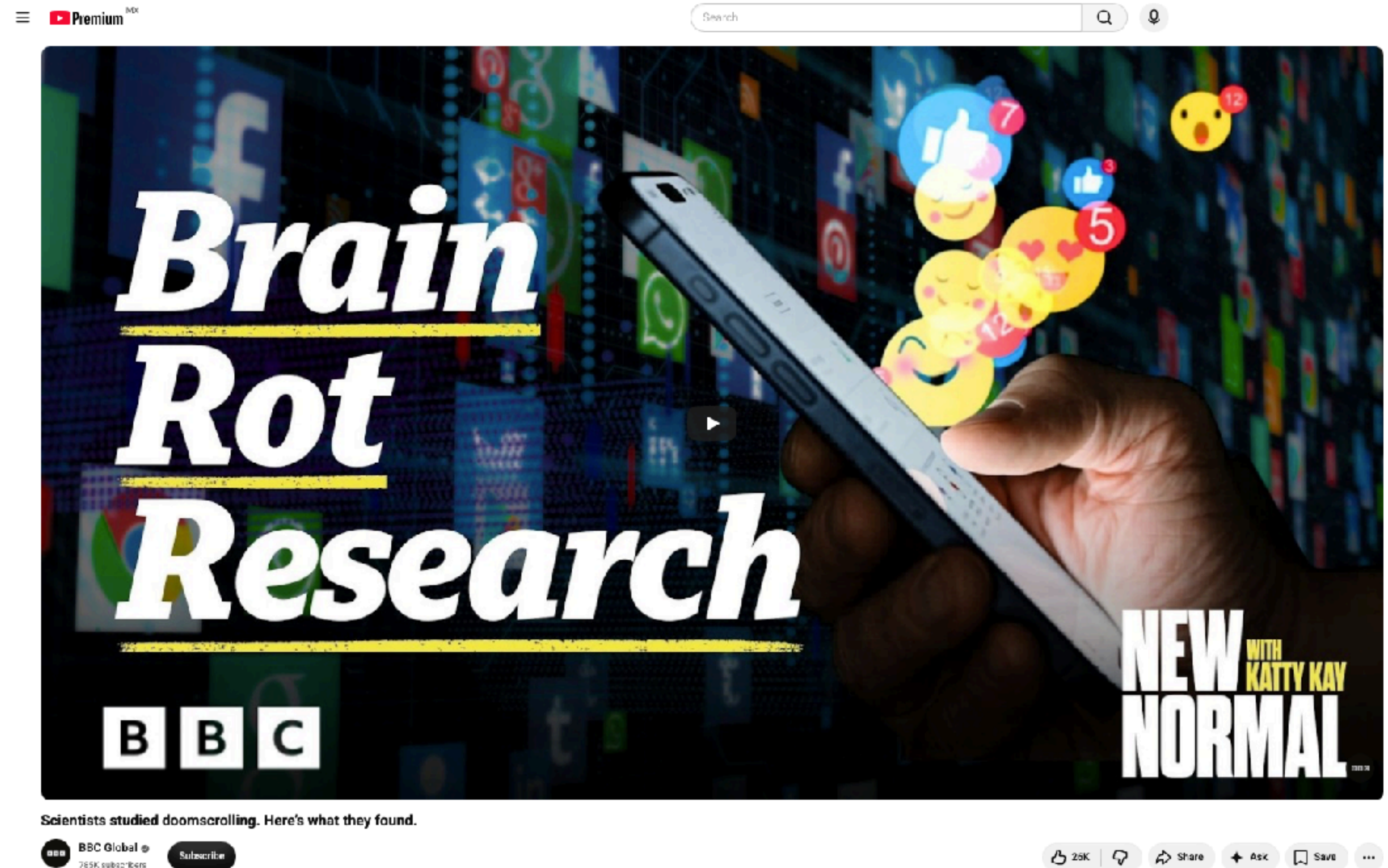
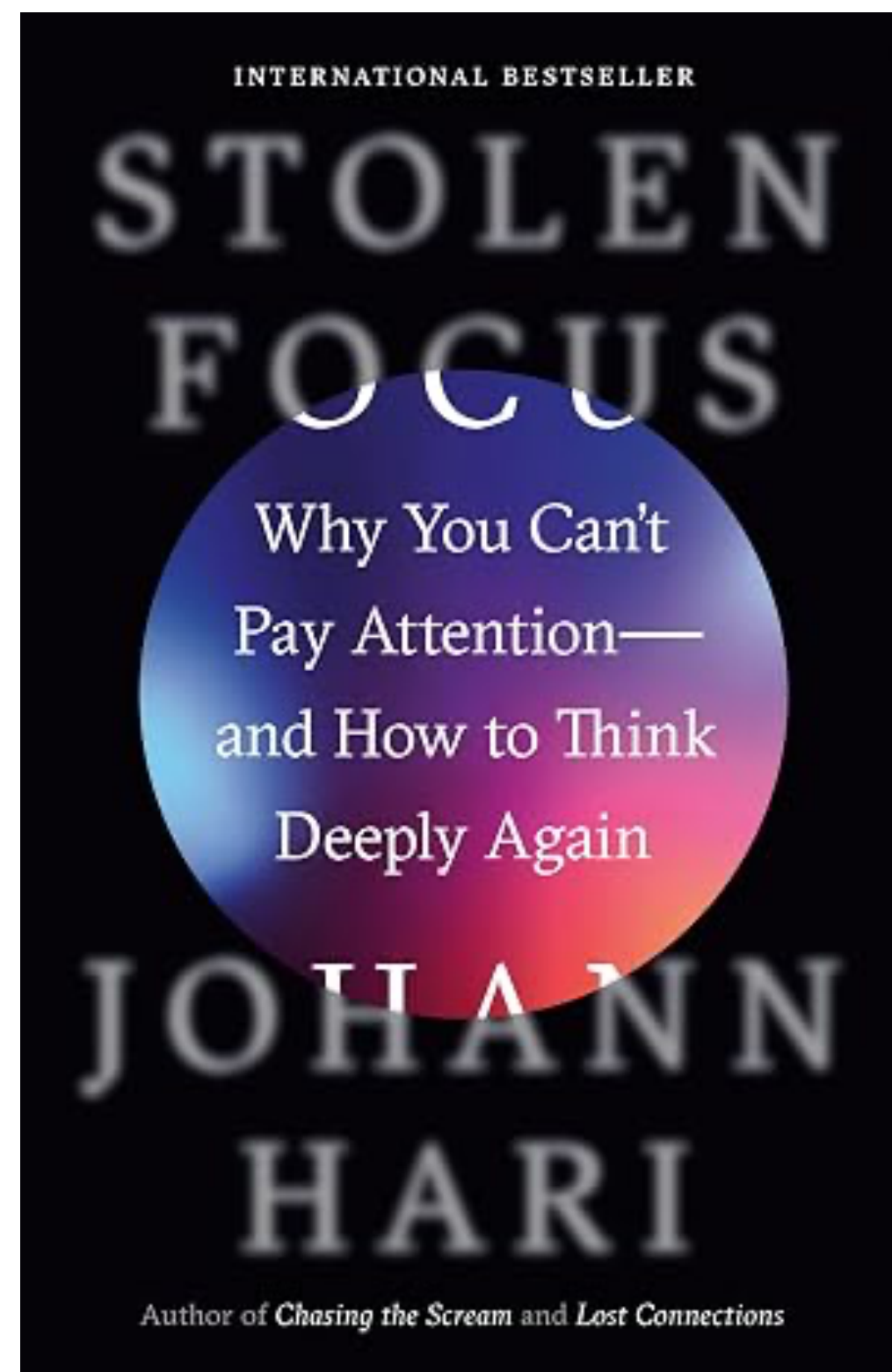
Jake Miller

Follow

4 min read · Jun 23, 2025



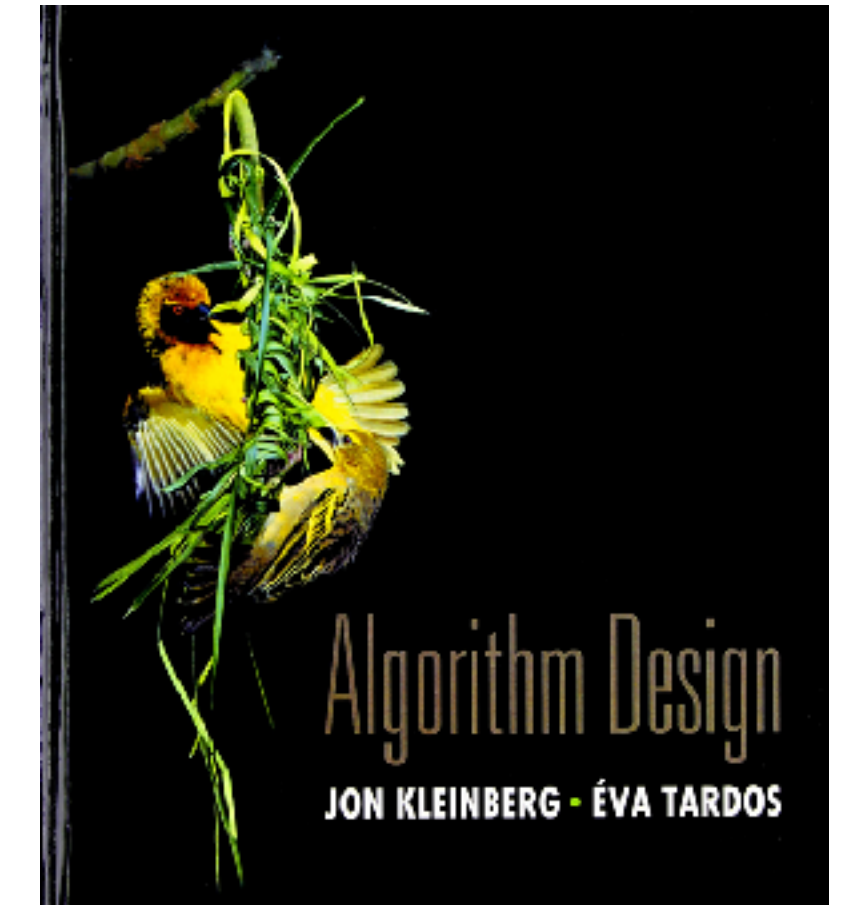
Recuperemos nuestra capacidad de atención



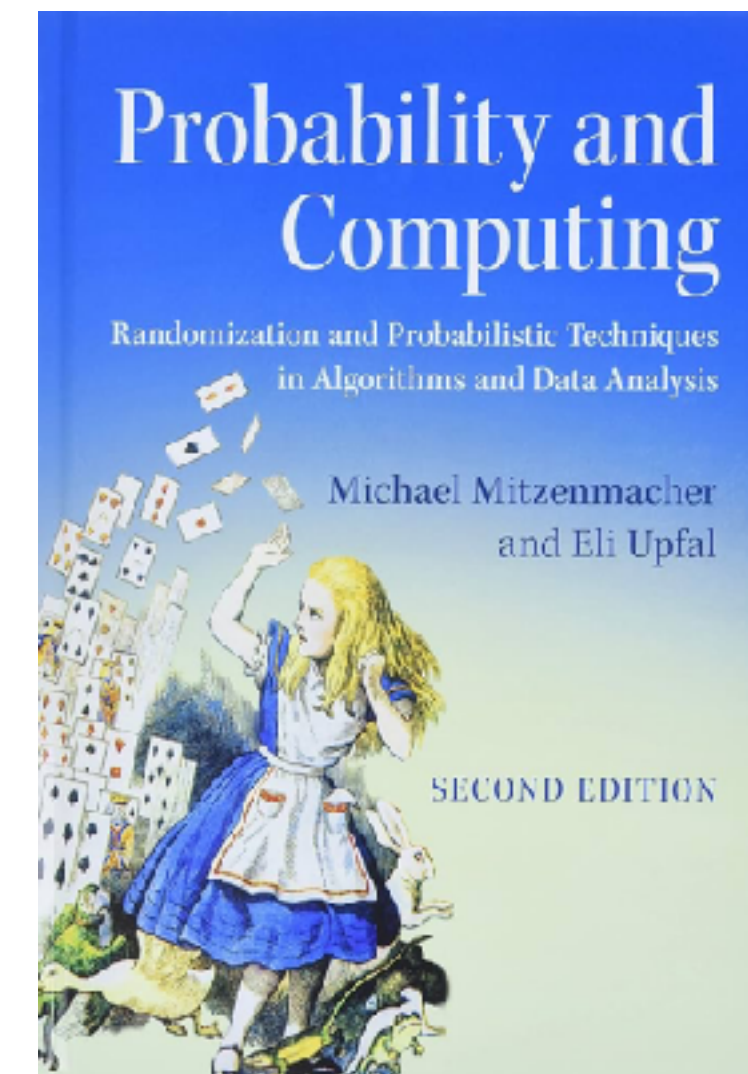
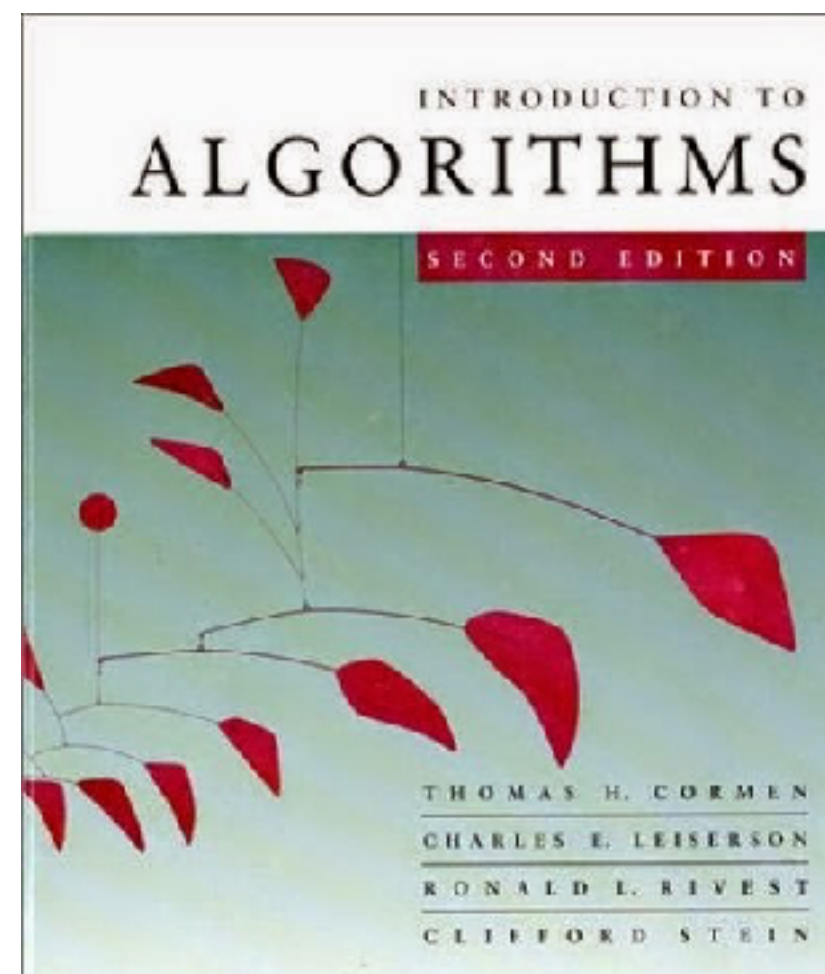
Material de referencia

- Principalmente: Algorithm Design. Kleinberg y Tardos

Slides: <https://www.cs.princeton.edu/~wayne/kleinberg-tardos/>



- Referencias adicionales:



Información del Curso

- Clases:
 - Martes y Jueves 11:30am a 1pm.
 - Salón C21.
- Ayudantes:
 - Ramses Hernández (ramsesh1m2@gmail.com).
 - Reinaldo Quispe (quispe.medrano23@gmail.com).
- Classroom del curso: <https://classroom.google.com/c/ODcyMDU4NTM4NTQy?cjc=dcfkhjsx>
- Código classroom: dcfkhjsx

Calificaciones

- Tareas. Individual o equipos de dos personas (dependiendo del número de estudiantes).
- 1er examen: Mediados de Septiembre.
- 2o examen: Finales de Noviembre/Inicios de Diciembre.
- Exposición final, en equipos; en Noviembre.
- Actividades en clase. Pueden dar puntos extras.
- Tareas 40%, Exámenes 30%, Exposiciones 30%.
- Política anti-deficit cognitivo (para el buen uso de la IA):

Promedio aprobatorio de exámenes es requisito para aprobar el curso.

